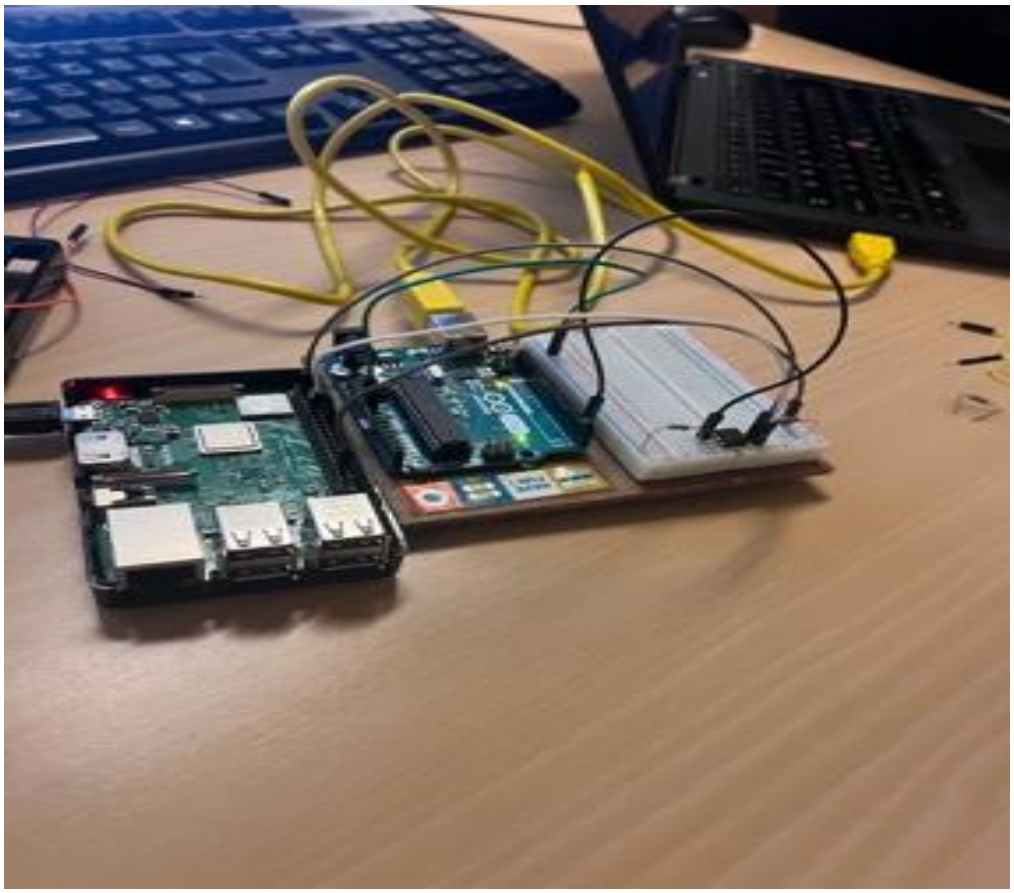


Laborationsrapport

Laboration 3
DT171G, Inbäddade system för Internet of Things

Författare: Omar Sultanzoy & Siye Abera, omsu2400@student.miun.se & siab2400@student.miun.se
VT26, år: 2 , 2026

Mittuniversitetet
MID SWEDEN UNIVERSITY

1 Sammanfattning

Uppgiften går ut på att skapa en fungerande seriell kommunikation mellan en Arduino och en Raspberry Pi med hjälp av en optokopplare. Arduino ska skicka minst 10 tecken, Raspberry Pi ska ta emot och visa dem på skärmen, och kommunikationen ska innehålla någon form av felhantering för att säkerställa att data överförs korrekt. Dessutom ska lösningen testas och analyseras, bland annat genom att undersöka hur hög kommunikationshastighet som är möjlig.

2 Innehållsförteckning

..... 1
1 Sammanfattning 2
2 Innehållsförteckning 3
3 Slutsatser 8
4 Källförteckning 9

Konstruktion

```
/*
  Författare: Omar Sultanzoy & Siye Abera
  Kurs: DT171G - Inbäddade system för Internet of Things
  Kursansvarig: Mikael Hasselmalm & Jan-Erik Jonsson
  Laboration 3
*/

void setup() {

  Serial.begin(300);
}

void loop() {

  Serial.write('A');

  delay(500);
}
```

Detta är Arduino-koden. I setup() startas den seriella kommunikationen med baudrate 300.

I loop() skickar Arduino tecknet **'A'** via seriell kommunikation och väntar sedan 500 millisekunder innan den skickar igen.

Det betyder att Arduino kontinuerligt skickar bokstaven **A** två gånger per sekund till den anslutna enheten till Raspberry Pi.

Sammanfattningsvis skickar Arduino kontinuerligt ett tecken, och Raspberry Pi tar emot och visar det.

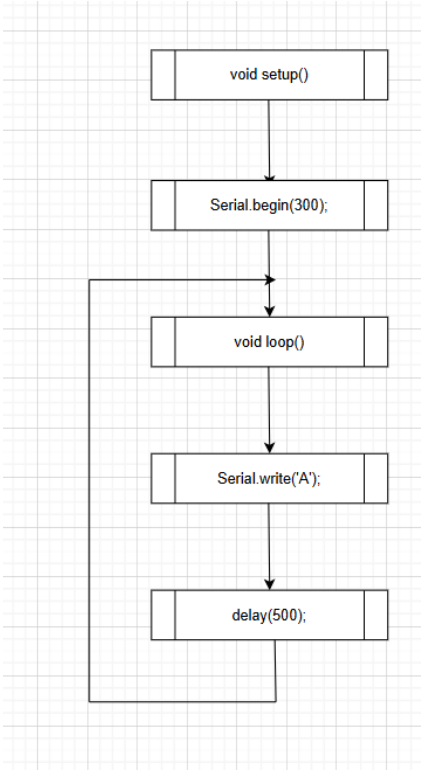
```c
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <wiringPi.h>
#include <wiringSerial.h>

int main() {
  int fd;
  if ((fd = serialOpen("/dev/ttyS0", 300)) < 0) {
    fprintf(stderr, "Unable to open serial device: %s\n", strerror(errno));
    return 1;
  }

  while (true) {
    while (serialDataAvail(fd)) {
      printf("data recv: %d", serialGetchar(fd));
      fflush(stdout);
    }
  }
  return 0;
}
```

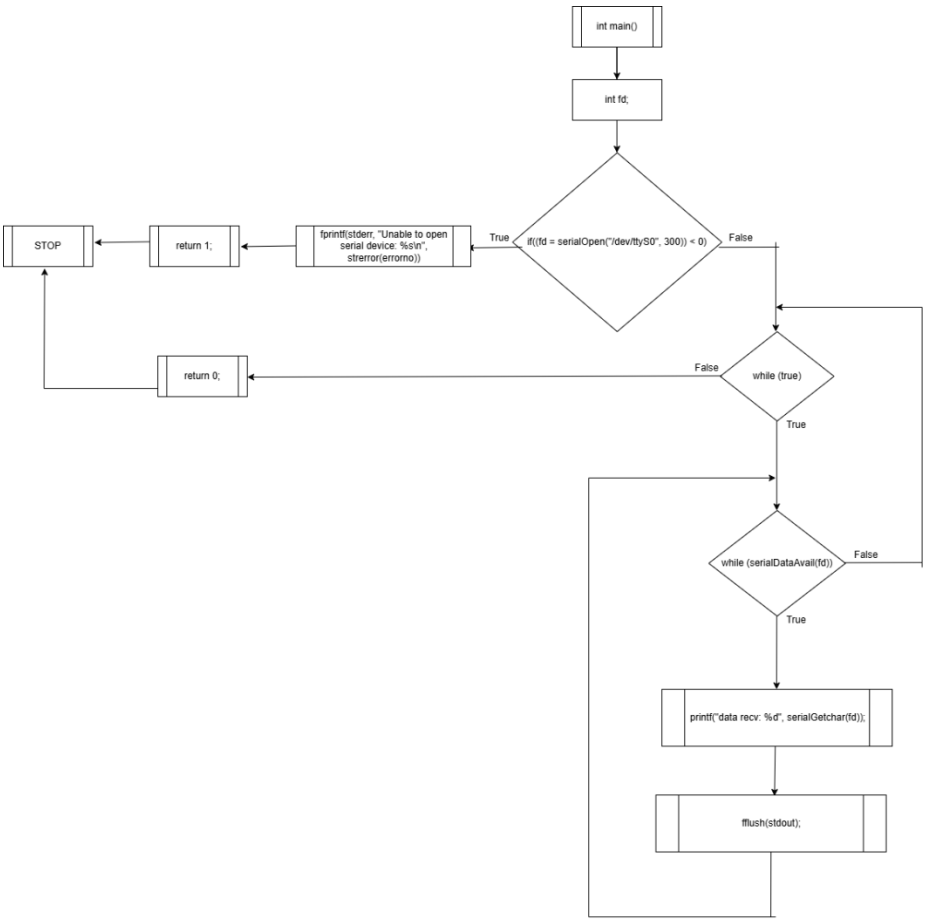
Programmet inkluderar bibliotek för seriell kommunikation (wiringPi och wiringSerial). I main() öppnas den seriella porten **/dev/ttyS0** med baudrate 300. Om porten inte kan öppnas skrivs ett felmeddelande ut och programmet avslutas.

Sedan körs en oändlig loop som hela tiden kontrollerar om det finns inkommande data. När data finns tillgänglig läses ett tecken med serialGetchar() och skrivs ut på skärmen med printf(). fflush(stdout) används för att säkerställa att utskriften visas direkt.



Figur 1: Flödesschema arduino kod

Figur 1 visar en enkel uppsättning av kod som är skapad i Arduino IDE för att skicka indata ”A” genom ortokopplaren och sedan till Raspbery Pi:n



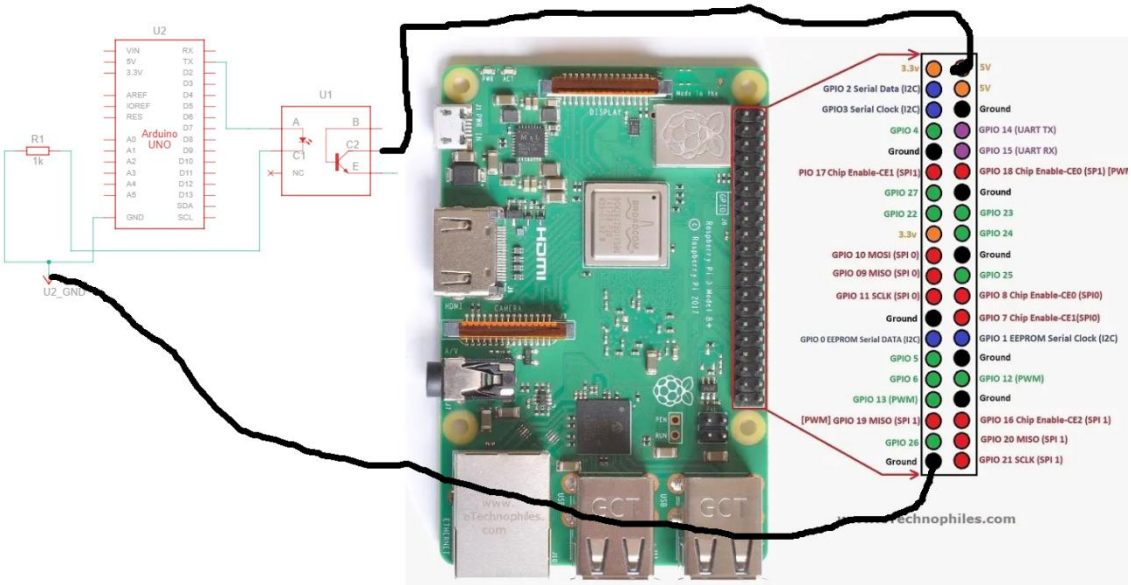
Figur 2: Flödesschema koden för Raspbery Pi

Figur 2 visar en enkel uppsättning av kod som är skapad i Raspberry Pi:n för att hantera datan som skickas via Arduinon, sedan displaya det i en terminal.

Komponentlista:

-1x Raspberry Pi
-1x Arduino UNO
-1x Optokopplare
-2x Resistorer
-Några kablar

Elschema:



Elschemat ser lite märklig ut, men låt oss förklara. I TinkerCad kan man inte lägga in Raspberry Pi:s, därför har vi kommit med denna lösning. I vänstra sidan ser man elschemat för Arduino:n och breadboarden med alla kopplingar. I vänsterledet ser man en bild på Raspberry Pi:s konstruktion, för pins:en iallafall. Vi har dragit svarta linjer mellan dessa för att representera kopplingen mellan dessa två.

3 Slutsatser

Genom uppgiften har vi lärt oss hur seriell kommunikation fungerar i praktiken mellan två olika system, samt hur viktigt det är att båda enheterna använder samma inställningar (t.ex. baudrate). Vi har också sett hur en optokopplare kan användas för att skapa elektrisk isolering och öka säkerheten i kopplingen.

Vi märkte att kommunikationen fungerar stabilt vid lägre hastigheter, men att högre baudrate kan ge problem om signalen inte är tillräckligt stabil. Detta visar hur viktigt det är med korrekt koppling och signalanpassning.

Sammanfattningsvis gav uppgiften en bättre förståelse för seriell kommunikation, felsökning och hur hårdvara och mjukvara måste samverka för att systemet ska fungera pålitligt.

4 Källförteckning

Här följer exempel på hur en källförteckning kan utformas enligt Vancouver-systemet. Den är automatiserad enligt metoden numrerad lista och korsreferenser. Radera denna text, samt ersätt källorna med dina egna.

[1] Midsweden University, ” Ansluta med fjärranslutning till Raspberry Pi i mitt laborationskitt.” https://canvas.miun.se/courses/8223/files/560405?wrap=1 Hämtad 2026-02-12

[2] Midsweden University, ” A short introduction to serial communication with Rpi written by Mikael Hasselmalm.”, https://canvas.miun.se/courses/8223/files/560406?wrap=1 Hämtad 2026-02-12

[3] Midsweden University, ”Download and Install”, https://canvas.miun.se/courses/8223/files/640907?wrap=1 Hämtad 2026-02-12